

南開大學
Nankai University

计算机学院

自然语言处理课程

文本分类实验报告

姓名：陈秋彤

学号：2311815

https://github.com/ApocalypseFallSpring/NLP_hw1

2026 年 10 月 7 日

目 录

1 实验概述 **2**

2 数据集与实验设置 **2**
2.1 数据集 . . . 2
2.2 数据划分 . . . 2
2.3 评价指标 . . . 2

3 Task 1：Bag of Words **3**
3.1 Binary Bag of Words . . . 3
3.2 Word Frequency . . . 5
3.3 实验结果与分析 . . . 6

4 Task 2：Word Embedding **6**
4.1 基于词向量的文档表示 . . . 6
4.2 Pre-trained GloVe . . . 7
4.3 Word2Vec on AG News . . . 8
4.4 Word2Vec on NYT . . . 8
4.5 实验结果与分析 . . . 9

5 Task 3：Fine-tune BERT **9**
5.1 BERT 文本表示与 Fine-tuning . . . 9
5.2 实验结果与分析 . . . 10

6 综合比较与讨论 **11**

7 实验环境与运行说明 **12**

8 结论 **12**

1 实验概述

本次实验聚焦 NYT 新闻文本分类任务，要求分别使用 Bag of Words、Word Embedding 和 BERT 三种文本表示方法，并比较它们在同一数据集上的分类效果。在实现过程中，重点需要理解不同方法是如何表示文本，以及表示方式的差异会如何影响最终的分类性能。

通过课堂学习，我们知道，Bag of Words 根据词语是否出现或出现次数构造文档特征；Word Embedding 先将单词表示为低维稠密向量，再通过平均词向量得到整篇文档的表示；BERT 则利用预训练语言模型对文本进行上下文编码，并通过 Fine-tuning 直接完成分类。在本次实验中，对于 Bag of Words 和 Word Embedding ，我们均使用 Logistic Regression 作为分类器，尽量让性能差异集中在文本表示本身。

同时，为保证不同方法之间具有可比性，我们在实验中固定相同的训练集、验证集和测试集划分，并使用 Accuracy 和 Macro-F1 作为评价指标。后续报告将分别介绍各方法的实现过程，并结合实验结果分析不同文本表示方法的特点及其对分类效果的影响。

2 数据集与实验设置

2.1 数据集

本实验使用了 NYT 和 AG News 两个英文新闻数据集。NYT 是主要分类数据集，每条样本包含新闻文本 `text` 和类别标签 `label`；AG News 在本实验中只作为训练 Word2Vec 的额外语料，不直接参与 NYT 分类器的监督训练。

其中，NYT 共包含 11519 条样本，分为 business、politics 和 sports 三类。其分布见表 1。

表 1: NYT 全量数据类别分布

类别	样本量	占比
business	1429	12.4%
politics	1451	12.6%
sports	8639	75.0%
合计	11519	100%

2.2 数据划分

我们首先使用随机种子 `42` 对 NYT 全量数据进行打乱，再按照训练集、验证集、测试集 $= 8:1:1$ 进行划分。之后所有实验都固定读取同一份划分结果，以保证不同方法之间的结果具有可比性。

表 2: NYT 数据划分（seed=42）

子集	样本量	business	politics	sports
train	9215	1157	1134	6924
val	1151	124	152	875
test	1153	148	165	840

2.3 评价指标

本实验使用 Accuracy 与 Macro-F1 作为评价指标。

Accuracy. Accuracy 表示全部样本中预测正确的比例：

$$\text{Accuracy} = \frac{\text{预测正确的样本数}}{\text{总样本数}}. \tag{1}$$

Macro-F1. Macro-F1 关注模型在不同类别上的表现是否均衡。对于某一个类别，首先计算 Precision 和 Recall：

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{2}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{3}$$

其中，TP 表示该类别被正确预测的样本数，FP 表示其他类别被错误预测为该类别的样本数，FN 表示属于该类别但被模型错误分类的样本数。之后，利用 Precision 和 Recall 计算该类别的 F1：

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{4}$$

对于多分类任务，需要分别计算每一个类别的 F1，再对所有类别等权平均：

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^{C} \text{F1}_c, \tag{5}$$

其中 C 为类别总数。

Macro-F1 指标对于本实验尤其重要，因为 NYT 数据中的类别分布并不均衡，sports 类样本明显多于 business 和 politics。此时，仅观察 Accuracy 可能会受到多数类表现的较大影响，而 Macro-F1 能够更加直接地反映模型是否同时对三个类别都具有较好的分类能力。

3 Task 1：Bag of Words

3.1 Binary Bag of Words

Bag of Words（BoW）的基本思路是，先从训练语料中确定一个统一的词表，再用词表中每个词在当前文档中的出现情况来描述这篇文档。这样，原本长度不同、内容不同的新闻文本，就可以被转换成具有相同维度的数值向量，并作为后续分类器的输入。

具体来说，我们首先对训练集中的新闻文本进行分词，并将训练集中出现的单词汇总成词表

$$V = \{w_1, w_2, \ldots, w_{|V|}\}.$$

本实验中最终得到的词表大小为

$$|V| = 128343.$$

词表可以看作所有文档共用的一套特征空间：词表中的每个词都对应向量中的一个固定维度。例如，第 i 个位置始终表示单词 w_i。因此，对于任意一篇新闻，都可以按照同一词表构造出一个 $|V|$ 维向量

$$\mathbf{x} = (x_1, x_2, \ldots, x_{|V|}),$$

其中 x_i 表示词 w_i 在当前文档中的出现情况。

不同 Bag of Words 的区别主要就在于如何定义这里的 x_i。本实验我们采用的是 Binary Bag of Words，也就是只需要判断一个词是否出现，而不考虑它出现了多少次：

$$x_i = \begin{cases} 1, & w_i \text{ 在当前文档中出现}, \\ 0, & w_i \text{ 未在当前文档中出现}. \end{cases} \tag{6}$$

例如，假设词表中的一部分为

$$\{\texttt{team}, \texttt{election}, \texttt{market}, \texttt{game}\},$$

如果某篇新闻中出现了 team 和 game，而没有出现另外两个词，那么它在这几个维度上的表示就是

$$[1, 0, 0, 1].$$

即使 game 在文章中出现多次，对应位置仍然只记为 1。因此，Binary BoW 最终保留的是“哪些词在这篇文档中出现过”。

在实际实现中，我们只根据训练集建立词表。验证集和测试集仍然按照这一固定词表进行向量化，如果其中出现训练词表中不存在的单词，我们选择直接忽略。这样做是为了保证特征空间完全由训练数据确定，避免在构造文本表示时提前利用验证集或测试集的信息。

之所以可以使用这种表示进行新闻分类，是因为不同主题的新闻往往具有不同的词汇分布。体育新闻中更容易出现与比赛、球队相关的词，而政治新闻中更容易出现与政府、选举相关的词。因此，即使 BoW 不保留词序，也能够保留较强的主题信息。Logistic Regression 在训练过程中进一步学习不同词特征与各类别之间的关系，最终完成分类。

Binary BoW 的优点是结构简单、实现直接，并且能够完整保留具体词汇的出现情况；由于向量中绝大多数位置为 0，也适合使用稀疏矩阵进行存储。但它同样存在明显局限：首先，词序信息被完全忽略；其次，不同词在表示中相互独立，无法体现词之间的语义关系；此外，词表越大，向量维度也越高。

核心实现. 下面代码对应 task1_bow.py 中的关键步骤：仅用训练集构建词表，再按 Binary 模式生成稀疏文档向量，最后用 Logistic Regression 训练。

```python
# 1) 只用训练集 token 建词表：词 -> 列下标
def build_vocabulary(tokenized_docs):
    vocab_set = set()
    for toks in tokenized_docs:
        vocab_set.update(toks)
    return {w: i for i, w in enumerate(sorted(vocab_set))}

# 2) Binary BoW：词出现记 1，OOV 忽略；输出稀疏矩阵
def vectorize(tokenized_docs, word2idx, mode="binary"):
    rows, cols, data = [], [], []
    for row_id, tokens in enumerate(tokenized_docs):
        counts = Counter(t for t in tokens if t in word2idx)
        for word, cnt in counts.items():
            rows.append(row_id)
            cols.append(word2idx[word])
```

```
            data.append(1.0 if mode == "binary" else float(cnt))
    return csr_matrix(
        (data, (rows, cols)),
        shape=(len(tokenized_docs), len(word2idx)),
    )

word2idx = build_vocabulary(train_toks)          # |V|=128343
X_train = vectorize(train_toks, word2idx, "binary")
X_val   = vectorize(val_toks,   word2idx, "binary")
X_test  = vectorize(test_toks,  word2idx, "binary")

clf = LogisticRegression(solver="lbfgs", C=1.0, random_state=42)
clf.fit(X_train, y_train)
```

3.2 Word Frequency

Word Frequency 与 Binary Bag of Words 使用相同的词表和整体处理流程，区别在于向量中每个位置记录的不再是单词是否出现，而是该单词在当前文档中的实际出现次数。对于词表中的第 i 个词 w_i，其对应特征可以表示为

$$x_i = c_i, \tag{7}$$

其中 c_i 表示 w_i 在当前文档中出现的次数。例如，对于文本 “apple apple banana”，如果词表中包含 `apple` 和 `banana`，则对应位置分别记录为 2 和 1。

与 Binary BoW 相比，Word Frequency 进一步保留了它在文档中的出现频率。从直觉来看，词频可能为分类器提供额外的信息，帮助分类任务。但实际上，词频并不完全等同于这个词的重要程度。一个词出现次数多，可能是因为文章本身较长，或者某些常见的介词、冠词被频繁使用。并且对于新闻主题分类任务，一些具有明显主题特征的词可能只需要出现一次，就可以提供足够强的分类信号了，此时继续增加频率信息也不一定能带来等比例的额外信息。

相比 Binary BoW 只考虑某个词 “是否出现”，Word Frequency 进一步考虑这个词 “出现了多少次”。

核心实现. Word Frequency 与 Binary BoW 共用同一词表与 `vectorize` 函数，仅将 `mode` 改为 `"count"`，把出现次数写入对应维度；分类器同样为 Logistic Regression。

```
# 同一词表，mode="count" 写入词频而非 0/1
X_train = vectorize(train_toks, word2idx, mode="count")
X_val   = vectorize(val_toks,   word2idx, mode="count")
X_test  = vectorize(test_toks,  word2idx, mode="count")

clf = LogisticRegression(solver="lbfgs", C=1.0, random_state=42)
clf.fit(X_train, y_train)
pred = clf.predict(X_test)   # 再算 Accuracy / Macro-F1
```

3.3 实验结果与分析

表 3: Task 1 实验结果

方法	划分	Accuracy	Macro-F1	log-loss
Binary BoW	train	1.0000	1.0000	0.0014
Binary BoW	val	0.9878	0.9687	0.0387
Binary BoW	**test**	**0.9887**	**0.9738**	0.0325
Word Frequency	train	1.0000	1.0000	0.0005
Word Frequency	val	0.9844	0.9641	0.0462
Word Frequency	**test**	**0.9861**	**0.9701**	0.0339

从表 3可以看到，两种 Bag of Words 表示在训练集上都取得了 1.0000 的 Accuracy 和 Macro-F1，说明模型能够充分拟合训练数据。在验证集和测试集上，两种方法仍保持较高性能，其中，Binary BoW 略优于 Word Frequency。在测试集上，Binary BoW 的 Accuracy 和 Macro-F1 分别为 0.9887 和 0.9738，而 Word Frequency 分别为 0.9861 和 0.9701；验证集上也呈现相同趋势。

除了 Accuracy 和 Macro-F1，我们还记录了 log-loss。log-loss 不仅关注最终预测是否正确，还考虑模型为真实类别给出的预测概率，数值越小表示预测结果越理想。可以看到，Word Frequency 在训练集上的 log-loss 更低（0.0005 vs. 0.0014），但在验证集和测试集上反而略高于 Binary BoW，说明更充分地拟合词频信息并没有带来更好的泛化效果。

结合两种表示方式来看，本实验中“词是否出现”已经能够提供较强的主题判别信息。Word Frequency 虽然进一步保留了词的出现次数，但这些额外信息还可能受到文本长度和重复用词的影响，因此没有带来性能提升。综合 Accuracy、Macro-F1 和 log-loss，Binary BoW 在本任务上的表现略优于 Word Frequency。

4 Task 2：Word Embedding

4.1 基于词向量的文档表示

Task 1 中的 Bag of Words 直接用词表中的不同维度表示不同单词，这种方法能够明确保留词语是否出现以及出现次数，但不同单词之间彼此独立，向量本身无法体现词语之间的语义关系。Word Embedding 则采用另一种表示方式：将每个单词映射为一个低维稠密向量，使具有相似语义或使用环境的词在向量空间中具有相近的表示。

实验中，我们统一使用 100 维词向量。对于每篇新闻，我们首先进行分词，再从对应的词向量表中查找每个单词的向量。假设一篇文档中共有 n 个能够找到词向量的有效单词，其向量分别为

$$e(w_1), e(w_2), \ldots, e(w_n),$$

则通过平均这些词向量得到整篇文档的表示：

$$e(d) = \frac{1}{n} \sum_{i=1}^{n} e(w_i). \tag{8}$$

因此，无论原始文本长度是多少，最终都会得到一个固定的 100 维文档向量，并将其输入 Logistic Regression 完成分类。

这种方法之所以适配新闻分类，是因为一篇新闻中，大量单词的语义分布通常与主题有关。体育、政治和商业新闻中出现的词虽然不完全相同，但同一主题中的许多词在词向量空间中具有一定的相似性。将这些词向量进行平均，可以得到一个整体上反映文章语义倾向的文档表示，之后再由 Logistic Regression 学习不同类别在这一向量空间中的分布差异。

与 BoW 相比，Word Embedding 将原本高维、稀疏的词汇特征压缩成了低维稠密表示，并能够体现一定的词语语义关系。但平均词向量也会损失部分信息：首先，平均操作仍然忽略了词序；其次，一篇文章最终被压缩成一个 100 维向量，一些具有较强分类作用的关键词可能会在大量词向量的平均过程中被削弱。

本 Task 下的三组实验我们采用完全相同的文档表示方法和 Logistic Regression 分类器，主要区别在于单词的 100 维词向量由不同方式获得。

4.2 Pre-trained GloVe

我们使用预训练的 100 维 GloVe 词向量来表示文档。GloVe 已经在大规模外部语料上完成训练，我们直接利用即可。

对于每篇 NYT 新闻，我们先进行分词，再根据 GloVe 词表查找对应的词向量。对于能够找到表示的单词，将其 100 维向量进行平均，得到最终的文档向量；不在 GloVe 词表中的单词则直接忽略。实验中，GloVe 对 NYT 文本的词汇覆盖率约为 0.98。

GloVe 作为预训练好的词向量，能够提供较好的词汇覆盖和较稳定的语义表示，但这些词向量并不是专门针对 NYT 分类任务训练得到的，因此也可以与后续实验形成对比。

核心实现. 下面代码对应 `task2_embedding.py`：加载预训练 GloVe，对每篇文档中能查到的词向量取平均（OOV 跳过），再训练 LR。

```
# 加载 GloVe：每行 "word v1 ... v100"
vectors = {}
with open("glove.6B.100d.txt", encoding="utf-8") as f:
    for line in f:
        parts = line.split()
        vectors[parts[0]] = np.asarray(parts[1:], dtype=np.float32)

# 文档 -> 100 维：有效词向量取平均；全 OOV 则用零向量
def docs_to_matrix(tokenized_docs, vectors, dim=100):
    X = np.zeros((len(tokenized_docs), dim), dtype=np.float32)
    for i, toks in enumerate(tokenized_docs):
        vecs = [vectors[w] for w in toks if w in vectors]
        if vecs:
            X[i] = np.mean(vecs, axis=0)
    return X

X_train = docs_to_matrix(train_toks, vectors)
clf = LogisticRegression(solver="lbfgs", max_iter=2000, random_state=42)
clf.fit(X_train, y_train)
```

4.3 Word2Vec on AG News

我们使用 AG News 中的 90000 篇新闻文本自行训练 Word2Vec。实际实现时，我们采用 gensim 中的 CBOW 模型，并将词向量维度设置为 100，`window`=5，`min_count`=2，训练 5 个 epoch。

CBOW 的基本思想是根据一个词周围的上下文预测当前词。在训练过程中，模型不断利用文本中的词语共现关系更新向量，因此经常出现在相似上下文中的词最终会得到较为接近的表示。这个过程只需要原始文本，不需要新闻类别标签。

得到 Word2Vec 模型后，我们再按照与 GloVe 相同的方法表示 NYT 新闻文档。与直接使用 GloVe 相比，这种方法的词向量完全由我们选定的新闻语料训练得到，因此可以观察在规模较小但同样属于新闻领域的语料上，我们自行训练 embedding 能达到怎样的效果。

核心实现. 先在 AG News 纯文本上用 gensim 训练 CBOW Word2Vec，再用上一节相同的平均池化把 NYT 文档变成 100 维向量并训练 LR。

```
ag_toks = tokenize_corpus(ag_df["text"].tolist(), "ag")  # 分词，无标签

model = Word2Vec(
    sentences=ag_toks,
    vector_size=100, window=5, min_count=2,
    sg=0,           # CBOW
    epochs=5, seed=42,
)
# 取出词向量表后，对 NYT 文档做平均（同 docs_to_matrix）
vectors = {w: model.wv[w] for w in model.wv.index_to_key}
X_train = docs_to_matrix(train_toks, vectors)
clf = LogisticRegression(solver="lbfgs", max_iter=2000, random_state=42)
clf.fit(X_train, y_train)
```

4.4 Word2Vec on NYT

第三组实验我们仍然使用相同的 Word2Vec 配置，但将训练词向量的语料由 AG News 换成 NYT。为了避免验证集和测试集信息参与表示学习，我们只使用 NYT 的训练集文本训练 Word2Vec。

后续的处理过程与上一组实验相同：先利用训练好的 Word2Vec 模型获得每个有效词的 100 维向量，再对一篇新闻中的词向量取平均，并使用 Logistic Regression 完成分类。

这一组实验与 Word2Vec on AG News 的主要区别仅在于词向量训练语料。AG News 的语料规模更大，而 NYT 训练集与最终分类任务的数据来源完全一致，因此两组结果可以用来比较不同训练语料对 Word2Vec 表示效果的影响。

核心实现. 与 AG 实验配置完全相同，仅把 Word2Vec 的训练语料换成 NYT **训练集**文本，避免用 val/test 学词向量。

```
# 仅用 NYT train 的分词结果训练 Word2Vec
model = Word2Vec(
    sentences=train_toks,   # 不用 val/test
    vector_size=100, window=5, min_count=2,
    sg=0, epochs=5, seed=42,
```

```
)
vectors = {w: model.wv[w] for w in model.wv.index_to_key}
# 后续 docs_to_matrix + LogisticRegression 与上一节相同
```

4.5 实验结果与分析

表 4: Task 2 验证集与测试集结果

方法	Val Acc	Val Macro-F1	Test Acc	Test Macro-F1
Pre-trained GloVe	0.9809	0.9529	**0.9835**	**0.9667**
Word2Vec on AG	0.9791	0.9503	0.9766	0.9501
Word2Vec on NYT	0.9791	0.9490	0.9792	0.9530

从表 4可以看到，三种词向量方法都取得了较好的分类效果，其中 Pre-trained GloVe 的表现最好，测试集 Accuracy 为 0.9835，Macro-F1 为 0.9667。相比之下，Word2Vec on AG 的两个指标分别为 0.9766 和 0.9501，Word2Vec on NYT 则为 0.9792 和 0.9530。验证集结果也基本保持相同水平，说明三种方法的表现较为稳定。

在两组自行训练的 Word2Vec 中，NYT 上训练得到的模型在测试集上略优于 AG News，Accuracy 高出 0.0026，Macro-F1 高出 0.0029。不过两者差距较小，说明仅改变 Word2Vec 的训练语料并没有造成很大的性能变化。相比之下，预训练 GloVe 的结果更加突出，这可能与其训练语料规模更大、词汇覆盖更加充分有关。

与 Task 1 相比，三种 Word Embedding 方法整体略低于 Binary BoW。词向量表示虽然加入了词语之间的语义关系，并将文档压缩到了 100 维，但平均操作同时丢失了词序和部分具体词汇信息。对于新闻分类任务，BoW 对关键词的直接保留具有较强优势。

5 Task 3：Fine-tune BERT

5.1 BERT 文本表示与 Fine-tuning

前两个 Task 都需要先将文本转换成一个固定的文档向量，再交给 Logistic Regression 完成分类。BERT 的处理方式不同，它直接使用预训练语言模型对整段文本进行编码，并在预训练模型后加入分类层，通过 Fine-tuning 完成端到端的文本分类。

与 Word2Vec 和 GloVe 中每个单词对应固定向量不同，BERT 中一个 token 的表示会结合其所在的上下文共同计算。因此，同一个单词出现在不同句子中时，可以得到不同的上下文表示。同时，BERT 在输入中加入了位置信息，使模型能够区分不同位置上的 token，并通过 Self-Attention 建模词语之间的上下文关系。相比 BoW 和简单的平均词向量，BERT 因而能够利用更丰富的上下文和位置信息。

实验中，我们按照作业要求，使用 `google-bert/bert-base-uncased`，Tokenizer 的最大序列长度设置为 `max_length=64`，并进行 3 个 epoch 的 Fine-tuning。学习率设置为 2×10^{-5}，batch size 为 16。

实际实现时，我们首先利用 BERT Tokenizer 将原始新闻转换为模型所需的 token 序列；对于超过 64 个 token 的文本进行截断，不足部分进行 padding。随后将处理后的输入送入预训练 BERT，并通过三分类输出层得到 business、politics 和 sports 三个类别的预测结果。

Fine-tuning 与前两个 Task 的区别在于，分类训练过程中不仅会更新最后的分类层，BERT 本身的参数也会根据 NYT 的标签继续调整。因此，文本表示和分类器并不是两个完全分开的阶段，而是在同一个分类目标下共同优化，使预训练得到的语言表示进一步适应当前新闻分类任务。

BERT 的优势是能够同时利用词语语义、词序以及上下文信息，不需要像 BoW 一样手动构造高维词表，也不需要像 Word Embedding 一样通过简单平均得到文档表示。但相应地，BERT 的训练和推理开销更大，而且本实验受到 `max_length=64` 的限制，对于长度超过 64 个 token 的新闻，后续内容会被截断。

核心实现. 下面代码对应 `task3_bert.py`：用 BERT Tokenizer 编码文本（截断到 64），加载带分类头的预训练模型，再用 Trainer 做 3 个 epoch 的 Fine-tuning。

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
enc = tokenizer(texts, truncation=True, max_length=64, padding=False)

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased", num_labels=3,
)

args = TrainingArguments(
    num_train_epochs=3,
    per_device_train_batch_size=16,
    learning_rate=2e-5,
    eval_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="macro_f1",
)

trainer = Trainer(
    model=model, args=args,
    train_dataset=train_ds, eval_dataset=val_ds,
    data_collator=DataCollatorWithPadding(tokenizer),
    compute_metrics=compute_metrics,  # Acc + Macro-F1
)
trainer.train()
```

5.2 实验结果与分析

表 5: BERT-base-uncased 微调结果

划分	Accuracy	Macro-F1
train	0.9967	0.9927
val	0.9774	0.9460
test	**0.9801**	**0.9562**

从表 5可以看到，BERT 在训练集上的 Accuracy 为 0.9967，Macro-F1 为 0.9927；在测试集上分别为 0.9801 和 0.9562。虽然测试结果相较训练集有所下降，但整体仍保持较高的分类性能，说明 Fine-tuning 后的 BERT 能够较好地适应 NYT 新闻分类任务。

与前两个 Task 的结果相比，BERT 并没有取得最高性能。Binary BoW 的测试集 Accuracy 和 Macro-F1 分别为 0.9887 和 0.9738，相比 BERT 分别高 0.0086 和 0.0176；Task 2 中表现最好的 GloVe 也取得了 0.9835 的 Accuracy 和 0.9667 的 Macro-F1，略高于 BERT。

这一结果可能与本实验的任务特点和输入设置有关。我们将 BERT 的最大序列长度固定为 64，因此对于较长的新闻文本，只能使用前 64 个 token，后续信息会被截断。其次，本实验是 business、politics 和 sports 三类主题分类，类别之间本身存在较明显的词汇差异，BoW 等方法已经能够利用关键词获得较强的分类能力。在这种情况下，BERT 更复杂的上下文表示并没有进一步转化为更高的分类指标。

6 综合比较与讨论

表 6: 各方法在 NYT Test Set 上的结果

任务	方法	Accuracy	Macro-F1
Task 1	Binary BoW + LR	**0.9887**	**0.9738**
Task 1	Word Frequency + LR	0.9861	0.9701
Task 2	GloVe Mean + LR	0.9835	0.9667
Task 2	Word2Vec-AG Mean + LR	0.9766	0.9501
Task 2	Word2Vec-NYT Mean + LR	0.9792	0.9530
Task 3	BERT-base-uncased	0.9801	0.9562

综合结果后，我认为本次实验最重要的认识不是“哪一个模型分数最高”，而是不同表示方法究竟保留了什么信息。

（1）复杂的表示并不一定带来更高的分类准确率。 Binary BoW 是所有方法中最简单的一种，却取得了最高结果。这说明对于 NYT 的三类主题新闻，关键词本身就是非常强的特征。只要任务能够被这些局部词汇信号很好地区分，简单模型也可能十分有效。

（2）BoW 与 Embedding 实际上做了不同的信息取舍。 BoW 的维度很高，而且稀疏，但每个词都被明确保留下来；平均词向量只有 100 维，表示更紧凑，并包含一定语义关系，但压缩过程中会损失词序和关键词细节。因此不能简单地认为“有语义的表示一定更好”，还需要看任务更依赖语义泛化还是具体词汇信号。

（3）BERT 的优势也受到输入和任务条件限制。 BERT 能利用上下文，但本实验只能输入最多 64 个 token，同时新闻主题分类本身并不一定需要复杂的语义推理。因此它没有超过 BoW。我认为这并不否定 BERT 的表示能力，而是说明模型需要在合适的设置和任务中才能发挥优势。

（4）Accuracy 与 Macro-F1 需要结合观察。 由于 sports 数量明显更多，各方法的 Accuracy 都很高，而 Macro-F1 普遍低一些。这提醒我，在类别不均衡的数据上，只看总体准确率可能不够，少数类别的表现也应该被单独考虑。

7 实验环境与运行说明

表 7: 主要实验环境

项目	配置
系统	Windows 10
Python	Anaconda 3.11
核心库	pandas, scikit-learn, nltk, gensim, transformers, PyTorch
BERT 训练	torch 2.6.0+cu124，RTX 3060 Laptop GPU
随机种子	42

主要脚本位于仓库 `code/` 目录，运行顺序如下：

```
python split_dataset.py
python task1_bow.py
python download_glove.py
python task2_embedding.py
python task3_bert.py
```

更完整的依赖和运行说明见仓库 `README.md`。

8 结论

本次实验让我从三个层次理解了文本表示。Bag of Words 直接利用词是否出现或出现次数；Word Embedding 将词映射到具有语义关系的低维空间，再通过平均得到文档表示；BERT 则进一步利用上下文生成动态表示，并通过 Fine-tuning 让表示适应具体分类任务。

最终 Binary BoW 在测试集上取得最高结果（Accuracy 0.9887，Macro-F1 0.9738），GloVe 是三种词向量方法中表现最好的，BERT 在当前 `max_length`=64 的设置下没有超过 BoW。

对我来说，本实验最主要的收获是：**文本表示没有脱离任务独立存在的“绝对最好方法”**。一个方法是否有效，取决于它保留的信息是否恰好适合当前任务。对本次主题鲜明的新闻分类而言，简单的关键词特征已经非常强；而更复杂的语义和上下文表示虽然具有更强的表达能力，也会受到信息压缩、输入长度和训练条件等因素影响。